AQA A Level Computer Science NEA

Platformer Game using OpenGL, written in C++ and GLSL

Candidate Name: Milo Griffiths
Candidate Number: 5057
Centre Number: 63261

Contents of Documentation:

- Analysis – Page 2
- Documented Design – Page 7
- Testing – Page 16
- Evaluation – Page 19
- References – Page 24
- Technical Solution

Contents of Technical Solution, in order of appearance:

- Important notice on structure of code
- Makefile, used in compilation
- Font lookup table generator (lookup_table.py)
- Example level files (e.g: level/test.level)
- Program entry point (src/main.cpp)
- Program source code (inc/bounding.hpp → src/timed_job.cpp)
- Shader source code (shader/button_vertex.glsl → shader/object_fragment.glsl)

NEA Analysis

Explanation of the problem / background to the project

I am interested in making a platforming game as I find them interesting and very fun on a personal level, but also because games with complex movement and high speed interactions are very popular. I feel I could make a contribution to this market by producing a game that combines some of my favourite aspects from games I have played in the past. In the game itself, I intend to have many movement mechanics such as sliding, dashing, grappling, with the intent of maintaining high speeds. In platforming games, areas with the fastest movement which require skill to traverse are the most fun for me, and movement techniques are always some of the most fun parts of modern games.

As such, the product I aim to produce will be a level based platformer game, with a focus on quick movement and a goal of getting fast times on levels.

Who the prospective users are

This game would be targeting people who are interested in fast paced games with lots of movement based content. Potential customers could be Speedrunners of existing games, or even potentially players of driving games, in which fast movement is essential.

Interview with client

My client is a friend of mine named Dan. His relation to the project is that he is a gamer like me, and we have shared interests in the kinds of games which I will attempt to emulate to some degree in this project. As a result he is interested in the game I will develop.

Me: What kinds of games are you most interested in?

Dan: I enjoy open world games and platformers.

Me: How much do you enjoy platformers or driving games?

Dan: I love a good platformer. I was a big fan of the game “My Friend Pedro” which is kind of a mix between a platformer and a shooter, with nice dynamic movement in 2D. Also an enjoyer of Minecraft parkour. Also BattleBlock Theater.

Me: Do you consider complex movement mechanics to be an important part of some games?

Dan: Yes. Or at least varied mechanics. Sliding, crouching, and ways to change your speed and how you move. Things that change your perspective like tilting in the air are also cool. Nice complicated, variable physics makes for more interesting movement.

Me: How much do you enjoy games with complex movement mechanics?

Dan: I'd say I do. Sometimes you can overcomplicate it slightly. Good physics allows for simple inputs but complex movement, and that's what I enjoy. Being able to control everything with only a few different keys is better than having lots of inputs.

Me: What features of movement do you find interesting in existing games (give examples)?

Dan: Games like SuperHot with interesting physics and mechanics gives the player options. Once again games like BattleBlock Theater, but lots of these are cases where the movement itself is only good because of how it interacts with the environment. Minecraft again, having things in the environment which change how you move is good.

Discussion of interview with client

I appreciate the feedback from Dan given during the interview. What I think will be important to keep in mind is specifically the use of only a few different inputs to create a fun and diverse experience. This will become something to consider much more when I get to the stage where I am able to work more on the movement techniques available to the player in the game after the bulk of the engine development is complete. As Dan also mentions however, I recognise I will have to be careful not to constrict the player's options at all while adhering to the former requirement. I will also need to invest lots of time in developing the physics system in order to make the movement techniques feel natural and fluid. This will likely involve creating an algorithm similar to the one I saw in Kasper Fauerby's paper "Improved collision detection and response" in order to handle the collision part of the physics engine, a problem I am aware as notorious for being hard to get right at times. Considering these points Dan brought up in the interview should lead to a satisfactory product at the end of the process.

After further discussion with the client, he has also suggested a type of object which would speed up the player when touching it, in order to make the movement more interesting. I agree with the client that this would add an extra layer to the movement within the game, so I will do my best to implement this feature.

Discussion of existing systems

In addition to the mechanics of the game, I have a specific style in mind as well. I am envisioning in my mind, a sort of neon aesthetic of bright lights and quite vibrant colours; the sort of which you would see in games such as Destiny 2 (in the Vex Network areas of the game), and in Ghostrunner (in the Cybervoid sections of the game). Both of these games feature movement techniques, with Ghostrunner having the fast paced aspect I really want to explore more.

As for the technical side of things, I do not plan to use an existing game engine to make this game. The game itself is not going to be quite as complex as a game like Destiny or Ghostrunner, to compensate for the fact I want to build all the algorithms required to construct a game like this (to an extent) from scratch. Obviously I am not making a new graphics API or programming language; the plan currently is to use the OpenGL API with the C++ programming language, as well as using the OpenGL Shader Language to write my own shaders. In addition to this, I will be developing this on Linux, and I do not plan to port it to Windows or Mac. I will also be using the OpenGL Mathematics Library to provide the vector and matrix types I will need for this project. I will also make use of the GLFW library to handle creation of windows, as well as their tools to detect keyboard and mouse input.

Additionally, these games both have systems (especially Ghostrunner) that you can use to share your fastest times and other statistics on certain levels with other players. There is potential for a system in which users can upload times to the internet, though this might be slightly ambitious. A system in which a user can see their own best times locally is achievable and I will try and do that.

I plan to implement a lighting model in order to augment the visual style of the game. Various lighting models exist, such as the Blinn-Phong lighting model which I believe will suit this project the best due to its simplicity and how inexpensive it is to implement in terms of performance. Other techniques such as shadow maps and effects such as bloom and HDR are possible to implement, and also may help with the style of the game.

For designing models, there are various tools out there which can help with this, but I believe Blender will be mos suited to this task. After researching it, I found that it has the ability to export model data in the Wavefront.obj file format, which is notably easy to parse compared to other formats. It also has the ability to triangulate faces of objects, which would allow them to easily be loaded into the program.

Modelling of the problem

I am going to need to write a game engine from scratch. So, I will need multiple systems in place in order to do things required to render the game. At a high level I will need to have a system with which I can easily load objects and have them rendered for me. Within this, I will need to have the ability to load models made in a program such as blender, as well as applying textures to models and other objects. I will need a system to compile shaders that I will have written myself and use them, and I will need to wrap this up in a neat interface to make the system of adding new objects to be rendered simple. I will also need a system to assign certain attributes to constructs in the game, in order to apply certain rules or physical properties to them.

With the basic underlying systems for loading and rendering objects, I will then need to create a simple system to produce new collections of objects, called levels, which will make up the core gameplay. I will need to find a way to create, load, and save levels, probably storing them in a special file format that I will have to write a parser for. Then, on top of this all I will want to wrap this up in a nice graphical user interface which the users themselves can use to select pre made levels, or levels which they have written themselves. The levels themselves will be structured in a certain way. All the levels will have a start and an end, between which there will be a course of various platforms and structures which must

be traversed in order to reach the end. These platforms and structures will have various kinds of behaviours that can be assigned to them, such as them being able to move and rotate.

I will also need to implement a physics engine in order to handle the movement of the player as well detecting collisions between the player and both moving and stationary objects, producing an appropriate response. This response could be an action being performed such as the starting of a timer, or changing the direction of the player when colliding with a solid object. I will be using the OpenGL Mathematics Library to assist with the mathematical side of handling and detecting collisions.

Detailed Numbered List of Objectives

1. Create a modular game engine
    1. It will be able to load models
        1. Load a model from a file generated in blender
        2. Store its vertices, normals, and texture coordinates
        3. Stores its various textures
        4. Generates a Vertex Array Object (VAO), and a Vertex Buffer Object (VBO)
    2. It will be able to compile, link, and use shaders
        1. Load the shaders from a file
        2. Compile them
        3. Link them
        4. Use them
        5. Store this data in an object, with a templated method to set uniforms
    3. It will have a main render loop
        1. Have all the objects in the scene stored in a container with methods to manage them
        2. Process input from the user
        3. Update the positions of all objects as well as the player
2. A system that will calculate the speed, position, and acceleration of every applicable object every frame.
    1. Using the OpenGL Mathematics library and GLFW:
        1. Take input from the user each frame
        2. Using the stored direction of the player, and the current speed and acceleration, as well as other factors such as whether or not the player is grappling or sliding, calculate the new position of the player.
            1. Much of this could be stored in a state structure, or a class with methods to update this.
    2. Then using the stored attributes of any other moving objects, calculate their new position and speed, etc. This will be simpler than calculating for the user as many moving objects might move only in a simple loop.
    3. Additionally, detect collision between the player and any nearby objects. If the player is trying to move in a direction and there is an object in the way, adjust the player's speed and direction accordingly.
3. A system to create levels and place objects.

    1. This should have the ability to place physical objects that the player can use to move around with.
    2. This should have the ability to define attributes on physical objects which affect how the player interacts with them.
    3. There should be a way to save levels into a file, that can be read when the game is loaded.
4. A graphical user interface.
    1. This should have a home screen.
    2. This should have a menu to select games to play.
    3. The various menus could be stored using a tree structure or an adjacency list.
5. Gameplay mechanics.
    1. Each level should have a start and an end.
    2. Each level should be timed, and this time should be displayed upon completing a level.
    3. The level should end early if a time limit is reached, to encourage faster movement.
    4. The level should terminate if the player falls off the level.
6. An events system.
    1. This should have the ability to store functions which are run each frame.
    2. These functions should have data passed to them about how long they have been running for, and the total time they will run for.
    3. This system should be able to be used with the level loader, to assist in defining behaviour for dynamic objects which move over time.

List and explanation of acceptable limitations

1. It might be optimistic to design a visual level editor for the user to be able to create new levels themselves. Level editing and creation will be possible to do, but it will likely not be a very user friendly system and would probably require defining the position and size of different objects manually, rather than dragging and dropping them visually. I could see a way to preview a level, where you could fly around and inspect different parts of the level as you define them, but I believe it is acceptable if this does not make it into the final product.
2. No music. While I am decent at programming, I am no composer. I think it is acceptable if there is no music in this game or even any sound effects, but I could see the latter making it into the final product if I can find somewhere I can get free to use sound snippets on the internet. I also do not want to deal with licensing and copyright issues if I were to take existing music from an artist.
3. No Windows support. I do not intend to support the Windows operating system, as I will be developing this in Linux. This decision is for both practical reasons (being that I don't have a development environment set up for Windows) and ideological reasons (I don't want to set up a development environment for Windows as I don't want to program on a proprietary OS).
4. No sophisticated player animation. If I was using a game engine, then this would definitely be something I would do, but given I am writing my own, it could be difficult to create a system to animate a player. This shouldn't affect the gameplay too much as it will be played in the first person, but I would rather not have any player body animation rather than a very bad implementation of some animation.

NEA Documented Design

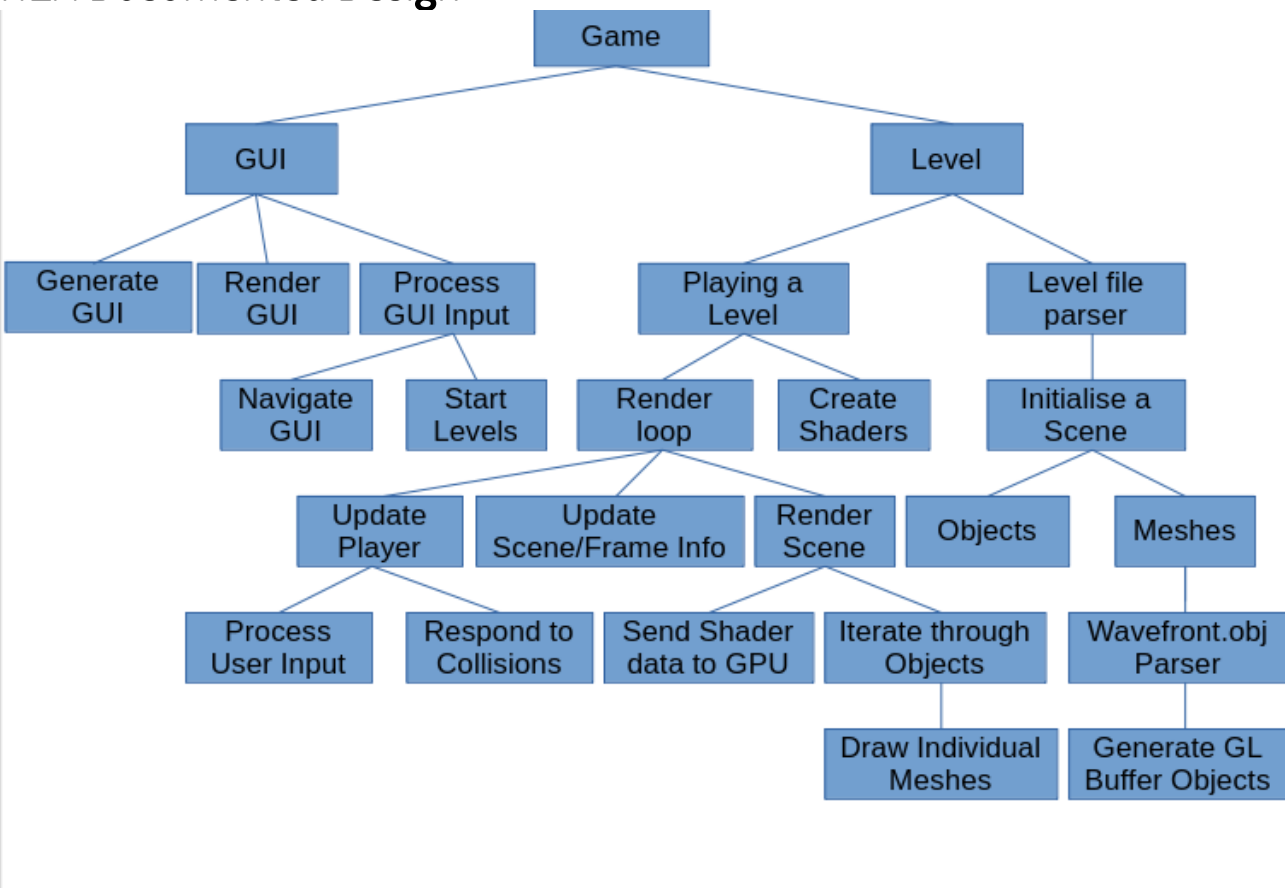


Figure 1: Program hierarchy chart.

Wavefront.obj parser

A wavefront.obj file is one file format used by the modelling tool Blender to store vertex data relating to a model that has been designed. An algorithm to parse this file format is required in order make the creation of new types of objects simple, avoiding the need to hard-code vertex data into the program. The wavefront.obj file format is relatively simple; it contains declarations of different vertices used in the model, as well as normals and texture coordinates, and then at the end it holds declarations of different faces, which are triangles consisting each of 3 triplets, each triplet holding 3 indices which point to the positions, texture coordinates, and normals declared earlier. The parser will work by reading each position, texture coordinate, and normal declaration and storing each in 3 different lists. It will then read each face declaration, and will generate triplets containing the required positions, texture coordinates, and normals, by using the values in the face declaration to index into the lists of values compiled earlier. An extra step to tokenise the face declarations will be required too, as the indices are separated by

forward slashes. Having extracted the vertex data from the file, this data is then able to be used to produce a Vertex Buffer which is used to send the data to the GPU to be rendered.

Collision Detection and Response

Collision detection and response form an important part of calculating and changing the movement of the player. These algorithms are required so that the player can collide with geometry in the scene and respond to it appropriately. The way in which the response works is by making the player slide along the geometry by projecting the players velocity vector along the plane formed by the normal vector of the geometry at that point. This preserves the player's current speed whilst also changing their direction. This also works with forces like gravity, in which case the projected vector across the plane you are falling on is very short or just has a length of zero. This whole system comes in two steps; detection and response. The response is what changes the velocity of the object being tested, while the detection checks if any collision has actually taken place, and if so, collects data about the collision such as the normal of the surface, the depth of the penetration, and more.

The detection will work roughly as follows: firstly, we will draw a bounding volume around the player. The shape of this will be a capsule as it most closely represents the shape of the player using only one simple construction. Next, we will have to iterate through each triangle of the mesh we are testing for collisions with in order to perform a test on each individual triangle. This test is performed by finding the closest point on the line segment across which a sphere is swept to produce the players capsule to each triangle, and then testing if the sphere at that point on the line with the radius of the capsule intersects with its respective triangle. If a collision has taken place then the maximum depth of the collision is measured and the normal of the penetration is noted down. This will then be collected with the rest of the data for each test, and values describing the collision with the whole mesh will be generated, likely by taking the maximum depth found, and potentially taking the average of the collision normals to generate a normal for the whole mesh.

The response will work by taking the collision information, as well as the current velocity of the player, and will use this to change the velocity of the player. The first step will be shortening to the velocity so that it ends within some miniscule value of the mesh it was going to collide with. The part of the vector which was cut off is then projected off the plane normal to produce another vector perpendicular to the surface at the point of collision. Then, this vector is added on to the shortened velocity vector and this resultant vector is then set to be the players new velocity vector. This whole process of detection and response can be repeated but the performance impact might be too large for this to be feasible, or the benefit of performing it multiple times might be minimal.

Scene Hierarchy

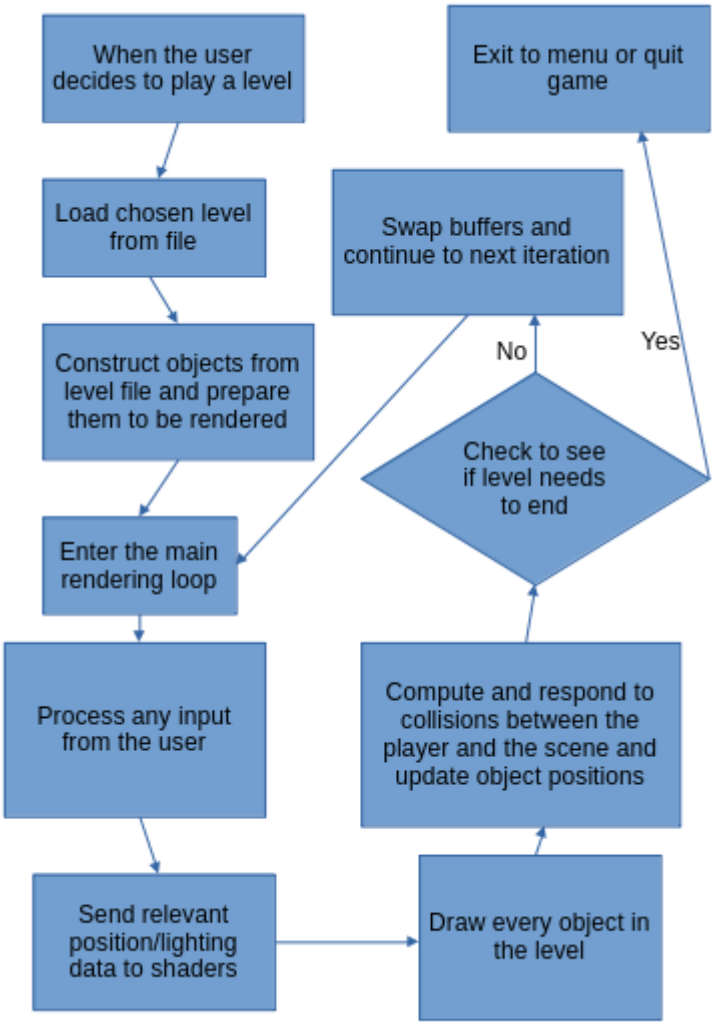
An important feature of this game engine will be the ability to cleanly load and store the physical objects in the game in some kind of data structure. The structure I have prototyped is a hierarchy of classes, beginning at the lowest level with the mesh structure, which stores the basic vertex data of physical object which has been loaded by the aforementioned model file parser. The next level is the object class,

which contains a mesh, as well as other data about the physical object more relating to its place and purpose within the game, such as matrices used to transform its position, behaviour to be executed upon collision, and other physical attributes which govern its behaviour in the game. Above this will be the bounding volume class, which I have decided will be an Axis Aligned Bounding Box (AABB), as it will be most simple to implement, and easiest to check whether or not a given object will be contained within it. At the moment I don't believe I will perform checks as to whether or not an object is contained within a volume for optimisation purposes, as the bounding volume is intended to be more of an abstract structure for grouping related objects. Above this will be the scene object, which as advertised will contain all the physical objects to be rendered in a given scene. I have arrived at this design as a result of a previous prototype in which I used variants to store different types of physical object (such as a cuboid or a sphere) within different types of bounding volume, also stored in variants, but despite the fact that cuboids will likely make up the main "building blocks" of a scene, working with variants was incredibly inconvenient due to the inability to at runtime extract the underlying type of the currently stored object within a variant, resulting in excessive, nested calls to std::visit. It was not pleasant to work with, so this current design which I have been prototyping is the result of this testing, and I plan to continue with this design through to the final project.

Render loop

The main render loop refers to the loop in which all the physical objects in a scene are updated and drawn. It must contain routines to draw the mesh of each individual object in each bounding volume, as well as run and respond to the collision detection routines in order to appropriately update the position of the player each frame. It will also be responsible for updating the events system, though the entire process regarding that system will be described in a following section. The current design will work by calling a method on a scene object to render its children, which in turn loops over every bounding volume it contains, which in turn render the meshes of their children. Additionally, as the render loop is also responsible for updating uniforms in the shader used when drawing the objects, the shader object will be sent as a parameter down through the render methods. The way in which those uniforms will be updated is as follows: currently, the only uniforms used are the various matrices used to project the positions of the objects onto the screen. These are partly generated via a method within the player class, which stores player data, which takes data from the player such as its position, and takes its rotation data from a structure marked with the storage specifier "extern" so it is visible to other translation units which need to access the players rotation data. Additionally, the individual transformations of each object must also be passed as uniforms, so those are set when the shader is passed to each individual mesh.

Figure 2: Render loop flow chart.



Level Construction

The current plan for the game itself is for it to be a platformer in which the player can play different levels and complete them, and then try to do them again with faster times. An obvious requirement therefore is the ability to load and construct a level using the scene hierarchy that will be created, the events system, and the input processor, which can then be completed by the player. The level will have additional mechanics to make it interesting, for example, it will have to have objects that have different behaviours on collision, or that alter your motion. Therefore a system to specify these levels must be created, which allows for both a way to define the basic structure of a level, but also different mechanics that would appear.

The current plan is as follows: I have prototyped a file format in which I can quickly create new structures contained within bounding volumes, each with their own characteristics like position and scale, as well as intended behaviours attached to them, for example, starting a timer upon colliding with them, or moving repeatedly between two points. Additionally, there is a level class, which will contain a scene to be rendered, as well as other data about the current level being played. When this class is

instantiated, there will also be an algorithm which takes a path to a level file as a parameter to the constructor, which will read this file and construct a scene object from it, whilst also attaching different extra characteristics through a system called "flags". There will be a flags parser, which will read in a list of characteristics stored after the object declaration in the file, which then modifies the created object by attaching collision behaviour or starting certain events with the events system, or by modifying other attributes.

The way the file will actually be loaded will be using a tokenizer from the Boost library, which can easily reduce a file in to a list of tokens across which an iterator can be passed. This will be transformed into a vector of strings, so that write operations can be performed upon the range of tokens more easily, as I plan to implement a method for including other files into a level file in a similar way in which C/C++ header files are pasted into other files via the #include directive. After building a simple prototype, I found it was easiest to do these kinds of operations after transforming the tokens produced by the tokenizer into a vector of strings. After the final list of tokens is produced, it will be looped through until a keyword indicating a declaration of an object is detected, which will then be responded to by passing the iterator being used to loop through the vector to a function which will parse all the following tokens which are part of the declaration. Following this, having retrieved an object from this function, the flags relating to this object will be parsed by another subroutine which takes both the iterator, and a reference to the object in question, which will allow for modifications to be made. This process will continue until the end of the file is reached, by which point the level will be fully constructed.

Input Processing

There will need to be a system to process inputs at various stages of the game in the final program, in order to allow the user to control the movement of the player, and also to navigate through the GUI. Currently, I have created a prototype of this system in which there is a single function which is run every frame while within the level, which in turns calls other functions which detect and respond to inputs on individual keys. This uses the GLFW library in order to detect key presses, and for the 5 movement keys currently implemented, return vectors which describe how the velocity of the player should be changed. The key functions return velocities to make it so that momentum can be carried by the player, allowing for more dynamic movement as the client originally requested.

The way these velocities are calculated involves storing a counter on the player class for each of the 6 directions the player can move in using the movement keys, relative to the direction the player is facing. The first 4 directions are forwards, backwards, left and right, which in essence are just counters which store how many frames the W, S, A, and D keys have been held for respectively. These counters are then read from by the key functions, and are used to sample a logarithmic curve, the result of which is multiplied with the corresponding direction vector, as well as the current frame time. This results in a smooth drop off in acceleration over the time the key is held down for. The way the final 2 directions will work will be where a counter is set for the up and down directions, and the user is able to jump only when the up counter is equal to 1. Jumping sets it equal to a different value. The down counter will increment for every frame the user's player is in the air, the value of which will be used to calculate the effect of gravity on the player. Whether or not the player is grounded (and in turn whether or not increment the down counter or allow the player to jump) will be calculated in the collision detection

process, where if no collisions are detected, or if the angle of the ground the player is standing on is too high (calculated via the dot product between the up vector of the player and the surface normal), grounded status will be set to be false. Any other key inputs required while playing the level can be programmed in as required with ease according to this model, though the ones listed are the only ones planned for now, in addition to a key to force close the program.

In addition to processing input during the game, inputs will also need to be processed while navigating the GUI. The GUI will be able to be navigated using only the mouse and the left mouse button to click on GUI elements. Therefore, the input processing function is quite simple. The input function will check if the mouse button is pressed, and will loop through interactive elements to see if the click occurred anywhere within the bounds of that element (there is no plan to have any elements which are any shape other than an axially aligned quadrilateral). If a click is within an element then that element's attached behaviour will be executed. In addition to this, I will implement a cooling off period after having a mouse click registered to avoid buttons being interacted with multiple times, and to avoid a button immediately being pressed after pressing another button to navigate to a new page.

Events system

I have identified that I will need a process which allows tasks to be started which run for a given amount of time, which may be used to do many things, such as call functions or update variables. The solution to this problem will be an events system, which will allow jobs to be started and to be run for a given duration, or indeed forever, executing some programmer defined behaviour stored in a function with a specific signature. This system will make heavy use of the std::chrono library within the C++ standard library to keep track of running jobs.

A job will be an instantiation of a job class, which will take a duration as well as a function of a specific signature as parameters to its constructor. This job object will then be able to be added via a method to a dictionary, mapping names of jobs to jobs themselves. A function will also be made which will take one of these dictionaries and will invoke the attached functions, passing 2 integers as parameters: the current time since the job was started, and the total time the job will run for. The specific signature of these attached functions as mentioned earlier will therefore be one of no return value, with two integer parameters. In order to pass extra data to these functions, I plan to implement some as lambda functions, utilising C++ language feature lambda captures to pass values, or using the std::bind function to bind additional pointers to other variables to the function so that those variables can be edited while the job runs, or both. This system is the event system I mention in the level construction section, and will be used in this way to modify objects within the scene over time.

Graphical User Interface (GUI)

A graphical user interface will be required in order to allow the user to choose levels to play, and to see results after completing or failing one, or to notify the user of important information. I have already described how I plan to implement an input processing function for this system, so this section will cover the planned implementation of the data structure itself and how it will be controlled.

I do not plan to use any external library to draw a graphical user interface, so all of the components will be implemented by hand, and as such will not have the same functionality as ones from popular libraries such as Dear ImGui.

The overall structure of the GUI will be a list of pages which can be displayed to the screen, each mapped to a name in order to allow pages to be selected within the program with ease. Each page is capable of storing any number of two different types of element: Labels, and Buttons.

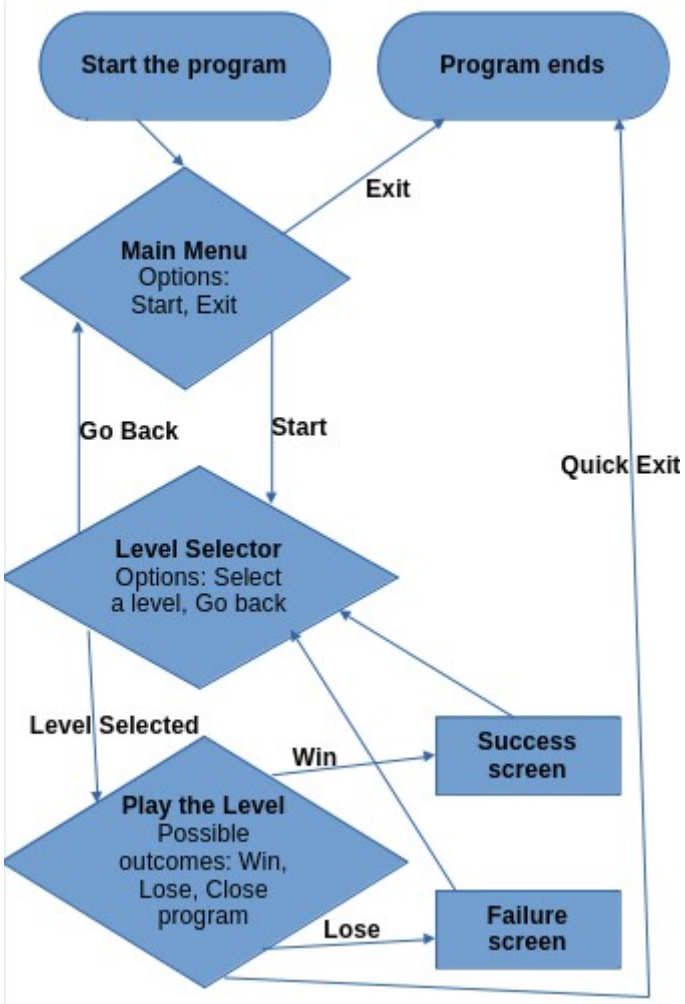
A label is conceptually very simple. It stores a location, some text to display, and the size of each character. I plan to implement this so that the location will refer to the bottom left hand corner of the first character "pane" to be rendered on the first line. As such, I would also like to have multi-line support, which could be implemented by iteration through the text to display, and incrementing a counter each time a newline character is found. The difficult part of the Label class will be implementing a text renderer from scratch. I have been working on a monospaced bitmap font, as well as a system which can, given a character, lookup a set of UV coordinates that correspond to this character on the bitmap image. This lookup table is stored in another class which contains details for a font, storing in addition to this table a special shader object which is used when rendering the text, and a method to generate a vertex array object which is called upon by the Label to generate vertex array objects which correspond to the individual characters in the supplied text to display. The lookup table is also currently generated programmatically by a python script to save time typing it out by hand. This system also means that I would be able to produce more bitmap fonts if I so desired, given that they are in the same format. As mentioned, the label will be given a font object upon creation which is used to generate vertex array objects for each character to be displayed. When the label is rendered, these array objects will be bound 1 by 1 and rendered in order. Within the font shader, no matrix transformations are applied to the text, and so the location of the text is supplied in the program in terms of its Normalized Device Coordinates, in the range -1 to 1 in all 3 axis (of course only the X and Y axis will be anything other than zero in this situation). This will likely be inconvenient to calculate in the program, so I plan on implementing a function to convert between pixels and normalized device coordinates to make this process easier. I will also look into allowing a colour to be supplied to the font shader, to increase the range of colours which can be used for the menu, improving its style.

A button will be likely simpler to implement as it will consist of only some information in regards to the dimensions of some box on the screen, as well as a function which in a similar way to the functions in the event system, will be of a specific signature, and can have parameters bound to them to allow extra data to be passed. A button will have its own specific shader, and as with the label, I plan for the size and dimensions to be specified in terms of normalize device coordinates, and so yet again a conversion function between pixels and normalize device coordinates will be useful. I also mentioned in the input processing section that in order for these functions to be called, a mouse click must be detected within the bounds of the button box. This will be simple and quick to calculate, involving just a few comparisons to be made in the X and Y axis to determine whether the function should be called. I will likely want text to be displayed on top of buttons to denote their use, but this will be done simply by creating both a label and a button in the same position, and layering the label on top. Calculations will be done within the

program to determine how these two elements should be position, but I may not implement a system in the future to do this automatically.

The GUI object will store the name of the currently focused page, and will automatically use this name to index into the dictionary which maps names to pages to call a rendering method on the currently focused one. In order to change which page is focused, a function will be used which takes a GUI object pointer (in order to make sure that changes to the state of the GUI persist) and the name of the new page, which simply edits the current page name attribute and sets it to the new name. This approach may not seem sensible, but I realise that the user will want to navigate the GUI using the buttons, and storing this behaviour as a function (which is essentially identical to how it would be if it were implemented as a method of the GUI class) allows it to have parameters bound to it so that it can be stored as behaviour inside a button object within a page.

Figure 3: GUI flow chart.



Lighting Model

A challenge later down the line will be improving the graphical quality of the game, and to arrive at a visual style which the client and myself believe suits the game. For this, I plan to write various shaders in order to produce lighting effects and colour the objects in a scene. Specifically, I will implement the

Blinn-Phong lighting model. This is a simple lighting model which is inexpensive to calculate and provides reasonably realistic lighting effects without impacting performance too heavily. The model consists of 3 light components: an ambient component, a diffuse component, and a specular component. The ambient component refers to the ambient lighting provided by small amounts of light always present in real life. This is an approximation of the work done by global illumination algorithms which are expensive and would not allow me to meet the performance targets which I have for the game. The diffuse component refers to the amount of light which hits and therefore illuminates an object based on the direction of the light. The specular component refers to the specular highlights which appear on illuminated, shiny objects.

The implementation of this will be quite simple. Many existing models employ the use of materials which contain values which control how the different components of the lighting model affect how the object is coloured in. I will likely not to this, and will only control the colour of objects via the intensity of lights around them (which will likely be specified in the level files), as well as the object colours. I believe this will produce satisfactory results so long as I tune the global ambient, diffuse, and specular values to suit the style of the game.

Another important factor to consider when designing and implementing this system, is the fact that actual emissive objects (lights) will not want to have the same shading rules as the non-emissive objects in the scene. Therefore, these will need to have different shaders applied, where the shader applied to the lights does not perform all the same lighting calculations as the shader used for the rest of the scene. Additionally, the light data for all the lights in the scene will need to be collected and sent to the regular shader so that all lights in the scene are accounted for. I plan to use a shader storage buffer in order to send this data to the shader on the GPU, which will contain an array of structures, all holding position, colour, and intensity values for the lights.

NEA Testing

Numbered list of tests corresponding to the numbered list of objectives

No.	Title	Method	Remarks
1	Loading a model	Create a mesh object in code and show that the number of vertices stored corresponds to the amount in the Wavefront.obj file.	
2	Rendering a model	Render this created model using the object shader to show it in action.	
3	Loading a shader from a file	Make a change in a shader to show that the shaders are being loaded from a file. Attempt to load a shader with an incorrect file path.	
4	Compiling and linking shader	Make a change in the shader that causes it to not compile. Make a change in the shader which causes it to fail to link properly.	
5	Using shaders and setting uniforms	Make a uniform which produces a visual effect, which is set in the program.	
6	Processing input each frame	Make the program print out a string when a specific key is pressed, and a different key when it is not pressed, in order to show that it works for both cases.	
7	Verify the movement of the player works as intended	Print out the velocity as the player moves along a flat surface to show the logarithmic nature of the player's speed.	
8	Verify the movement of objects works as intended	Show that an object with the "moving" flag moves correctly between two stationery points.	
9	Show collision works on simple objects	Collide with a normal platform, and jump on it. Collide with a moving platform, and jump on to it.	
10	Show sliding works on slanted objects	Collide with a slanted object with a slope angle above 45 degrees to show the sliding behaviour. Collide with a similar slanted object with an angle less than 45 degrees to show sliding movement on an object like that. Jump on a slanted object.	
11	A high velocity collision example	Fall from a large height and show that the player does not clip through the geometry.	
12	Show collision works on complex geometry (e.g. Suzanne)	Walk around on a torus and an imported blender monkey (Suzanne) to show how collision works on object with more complex geometry.	Movement can be slightly "Jittery" with these models.

13	Add a level while recording to demonstrate its functionality	Write a basic level, and show how it appears within the level selector, and play it.	
14	Demonstrate all different flag types on the object	Make a set of objects which each have one of all the different flag types: start, end, moving, rotate, accelerate, light. Show that all of these work as intended.	
15	Demonstrate includes	Make a new file with a single object, and include it within a level file, and show the object appear.	
16	Show in the program that the number of objects in the scene is correct	Count the number of objects in a level file and print the number of objects within the whole scene.	
17	Demonstrate creating a label to display on screen	Make a new label with some sample text, showing the use of font, font size, font color, and the pixel casting function to change the location of the label.	
18	Demonstrate creating a button with custom behaviour	Make a button which when clicked executes a function which prints text to the screen.	
19	Demonstrate linking a button to a new page	Make a button with a bound function which changes the state of the GUI to display a blank page.	
20	Demonstrate how adding a new level file adds a new button to the selector	Show the level file appear in the level selector. Show the behaviour which occurs when too many levels are added, using the copy and remove scripts.	
21	Show the functionality of start and end objects	Play a simple level.	
22	Show the kill floor	Jump off the map.	
23	Show the time limit ending the level	Start the level and wait for the time limit to be reached. Show the time limit in the level file to test that it corresponds correctly.	
24	Show the success and failure screens	This will be done as part of the last 3 tests.	Sometimes an incorrect time is displayed, cause

			unknown.
25	Create a new job which runs every frame	Make a job which prints some text every frame.	
26	Create a new job which uses the time data	Modify the job so that it outputs the time data supplied to it.	
27	Show contents of job list after starting level	Show that the number of jobs changes correctly when the level starts.	

Links

Full testing video: https://youtu.be/lD0WUPZtYH8
Video of a level completion: https://youtu.be/Xe4Upf_EHMs

Testing Notes and Conclusions

The testing appears to be fairly conclusive. The results are that the program works correctly and produces the intended result for nearly all of the tests run. There are however some important points and problems to consider. The first is that sometimes, the time displayed on the ending screen is incorrect. The time displayed can sometimes be the result from the previous successful attempt, and can even be displayed if the user fails the level. The cause for this is unknown, as restarting the program or my computer can fix it completely. It can be seen happening during the testing video once at 50:55. I suspect that it could be due to some memory issue, as other issues have been in the past, however it would be very difficult to track down due to the complexity of the codebase, and so I believe it is best to accept this error given that it can be easily solved temporarily. I may look into finding a solution at a later date. The second main issue is that the movement of the player when colliding with certain objects is inconsistent. When colliding with the underside of tilted surfaces is sometimes possible for the camera to clip into the geometry. The fix for this issue is contained within the function `sphere_triangle_collision` in the file `src/physics.cpp` however this fix only computes an approximation of an offset which should be applied to the position of the sphere placed on the player which is used to detect a collision. It does not fix the root cause of the problem. Additionally, collision with objects which rotate over time using the "rotate" flag is heavily inconsistent and collisions with sides of rotating objects which move toward the player do not produce the correct results.

The list of tests was produced by referring back to the numbered list of objectives contained within the project analysis. I believe the tests contained within this list show well the program in action, and all the main aspects of it. The tests are a summary of the most important parts of the list of objectives, and the parts for which it is feasible and sensible to test.

NEA Evaluation

Assessment of the Numbered List of Objectives

In this section I will refer back to the detailed numbered list of objectives contained in the analysis of the problem. To begin, I will discuss point 1: Create a modular game engine. Now referring to specifically point 1.1 on model loading, I would argue that this objective has largely been met, as there is indeed an algorithm present which loads vertices from a blender file using the Wavefront.obj format, successfully stores all the vertex data, and generates a Vertex Array Object (VAO) and a Vertex Buffer Object (VBO) from this data. However, point 1.1.3 has not been met. The algorithm does not have the capability to load textures from a blender file. The main reasons as far as I can see are as follows: Firstly, I did not have any textures that I wanted to load to use to decorate the levels. I felt at the time like the game did not need textures in order for it to look a certain way. Secondly, I was unable to find clear information when writing this part of the program on exactly how this is done. These are both failings in hindsight, and I believe the program would have been better if I did invest time into developing this system more. Additionally, I was not aware of procedurally generated textures, which could have fit the aesthetic better than something I could have created in a third party program. If I did this again, I would implement such a system. Continuing from model loading, I will now discuss point 1.2 on compiling and using shaders. This system as all in all been a success as all of the sub points within this section have been met. The shaders are loaded correctly, and the ability to report errors is also demonstrated in the testing video. There is also a clean system to update shader uniforms to set variables within specific shader programs. Templating the methods in this system was likely not necessary, however whether or not I did template these methods likely would not affect at all the end result of the program. What is not contained within the list of objectives which I added in hindsight was the ability to create Shader Storage Buffer Objects (SSBOs) which are able to hold a large amount of arbitrary data which can be sent to the GPU. This was implemented late on in the program, and it was very useful for implementing the lighting model I used. This I believe is an improvement over the original system of just using uniforms. Next, I will discuss point 1.3 on the main rendering loop. While the sub objectives in this section have certainly been met, I believe that this section should have included more detail as to what was actually desired. Additionally, these objectives have manifested in the final program potentially differently than was originally envisaged. I am mainly referring to the fact that there is not one main render loop, but rather two: one of them is the loop for the GUI, which processes input from the user, and can start another loop contained entirely within a Level object when the level is played.

Now I will move on to main point 2, on the movement of objects in the scene and the player. To begin, I will first discuss point 2.1. This section has largely been completed, input is indeed taken from the player every frame, and the players velocity is updated accordingly (not yet taking account for collisions; this is contained within point 2.3). Additionally, this player data is stored in a player class, though unfortunately data about the orientation of the camera is stored in a separate structure in order for it to be accessible to the GLFW mouse callback function. This is unfortunate, though I don't believe there is any other way of having it so that the callback function can edit this data other than having it accessible as a global variable. Additionally, grappling has not been implemented, and nor has sliding in the way I originally envisaged. This I believe is one of the greatest failings of the program. The movement could have had

these extra layers of complexity, and it doesn't, and I think it is possible to argue the game is missing something as a result. On the other hand, there are a few reasons why I did not implement these systems. The first is that this would have taken a very long time to implement. When I got to the point at which it would have made sense to begin developing this system, I had already developed a test level which be seen in the testing video, and both the client and myself, as well as others had been playing it and enjoying the game in its current state. I decided that it would be too ambitious given the time I had left to develop the project further in this way, and that I would just leave the game as a simple platformer, maybe with room to add this feature later. I regret not implementing this feature, and I would definitely have spent the extra time it would have taken to implement it if I did this project again. I may well do this later. Despite this failing, the client is pleased with the project regardless and understands that the game was already an ambitious endeavour. Point 2.2 has been fulfilled also, as objects have the capability to move or rotate over time, or both, however there is not quite as much calculation involved as was originally perceived. This is not a failing, just that this turns out to be a task less computationally expensive than it is logistically challenging. Point 2.3 has been fulfilled largely, and collision detection and response is implemented, although as mentioned in the testing, it does not work for objects which rotate over time, and sometimes bugs can appear with static rotated objects. This is something that I should have invested time into fixing.

Next I will move on to point 3, on the system of creating levels and defining objects. This entire section has been implemented in its entirety, and all 3 sub points, 3.1, 3.2 and 3.3 are implemented in the game. There is however, in my view, areas for improvement over what is described in the list of objectives. It would have been nice if the ability to define levels, produce objects, and add behaviour to those objects was visual rather than text based via editing a file. Whilst editing a file is simple, a visual system would have been ideal. This however would have been a very large endeavour that would have taken too much time to develop given the time frame. I can see myself in another project however, which might use a similar system, implementing a visual scene editor if there were not quite so many other aspects in this specific project.

The next point to discuss is point 4, which describes the graphical user interface required by the program. There could have been improvements in how this was detailed as less of the actual content of the pages was described in the list of objectives, so the technical details are more difficult to evaluate. Despite this, I believe that objectives 4.1 and 4.2 have been fulfilled, as there is indeed a main menu, which in turn leads to a different menu, which has many buttons which each start their own levels. Point 4.3 is more difficult to assess, as pages were certainly stored in a list, however the way in which they were linked is more complicated. They did not directly hold some kind of reference to each other, but rather hold button elements, which hold functions, which when executed change the state of the original GUI so that the new desired page is being displayed. In that regard this does behave somewhat like an adjacency list, or a list which stores connections for each node in a graph structure. While the solution which I ended up producing may not quite have been what I imagined when outlining the objectives for the GUI, it behaves in a similar way, but most importantly produces satisfactory results. However, more generally I believe more work could have been done on the GUI, such as implementing more kinds of elements. One that would have been useful would have been some kind of drop down menu, or some kind of scrolling mechanism to allow the user to store an infinite number of displayable levels. There is a limit to the number of levels displayed which is shown in the testing video, and I believe this is a

limitation even though the case in which this happens is handled sensibly by the program. A scrolling mechanism or a drop down menu, or any element conceivable which can hold an arbitrarily large number of interactive items would have been preferable to the solution I ended up creating.

Moving on, the next point to discuss is point 5, on the gameplay mechanics. This point is designed to outline a number of objectives for what the game should actually look like when the user plays and interacts with it. This section has been met in its entirety; 5.1, 5.2, 5.3, 5.4 have all been met and are tested in the testing video. I would argue that the gameplay mechanics are generally very complete and result in an experience which makes sense. There is however great room for improvements to be made. Whilst the gameplay mechanics could be considered very complete, especially in relation to the numbered list of objectives, there are obviously additions which could be made to make it more interesting. One system which I think could have been interesting to add in hindsight could have been a system of checkpoints, which must be reached in order to complete the level. This could have been good in order to force the player to take more difficult paths, or to add an extra sense that the user is making their own path through all the checkpoints as they play the level. This would have increased the difficulty of the game, and added an extra layer to the gameplay, and is something that I definitely would do if I started the project again.

Finally, I will discuss point 6, on the events system. This system is integral to many aspects of the program, and has been very useful throughout the development of the game. It sees use in the level creation for updating objects over time, as well updating per frame information, which is essential for making sure the movement of the player is consistent as frame rate changes. As such, I believe this system has been implemented fully as points 6.1, 6.2, and 6.3 describe. I would like to comment however, that despite the fact it has been implemented and used in things like the level loader as the objectives describe, it was quite arduous to find a method to get it to work due to the way in which the program was structured. This is not necessarily a comment on the functionality or usability of the events system, but rather a comment on the fact that the program had grown so complicated by that stage of development that it was very difficult to implement this feature without logical errors, where the effects of the timed job would not take effect, or runtime errors such as segmentation faults. I would revise the structure of the program to produce something far more simple if I were to do this project again, so as to avoid situations like this.

Additional note on the Lighting Model

The lighting model in the final product is not as impressive as I originally envisaged. The renderer does not have the ability to produce shadow maps for physical objects, nor does it have post processing effects such as bloom and HDR. This I believe is disappointing, though is not as much of a failing as the lack of movement techniques could be seen to be. These techniques are not easy to implement, especially shadows, which are a notoriously complicated problem in renderers which use rasterisation to draw objects. In the end, the lighting model, whilst not as impressive as described, did still look fairly visually pleasing. I was of course in contact with the client throughout development, and he does not have any problems with the lighting model, which was also a factor in deciding to not develop it further.

Feedback from the Client

These are a set of questions which I asked to the client following the completion of testing and assessment of the numbered list of objectives above.

Are you satisfied with the final product?

From the client: "Very happy. Your client has been telling his missus how great the product is."

What parts would you say have been successful?

From the client: "I like the difficulty, and I am a fan of the acceleration pads. I also think the collision is very solid, and the fact that you can load blender models is very great. The level design is also really well implemented. The language you created to make the levels is very successful."

What do you feel could have been added?

From the client: "Sliding or Spiderman-esque swinging like grappling would have great. The level you demonstrated the game with was quite difficult and something easier as a beginning would have been nice. It was like the Dark Souls of platforming. A difficulty system with different time limits for each level would have been nice in order to lower the entry bar."

How well do you feel the list of objectives has been fulfilled?

From the client: "[in reference to the lack of textures] I probably prefer it textureless anyway, and I prefer a more geometric look for games anyway, so I am not that bothered about it. [in reference to the lack of sliding and grappling] I don't think this is a failing of the project, but it would have been nice to have as it would have allowed for greater variation in the kinds obstacles which could have been created, and a greater range of gameplay, but the base gameplay is very nice as it is. The collision is successful, and the jittery movement with complex objects is fine so long as there is no clipping. It isn't perfect in triple A games so I wouldn't expect a perfect implementation here. I'm happy with the level construction. The only problem would be that I wouldn't know how to do it as for a client it seems quite complicated. The first two GUI objectives have been met, and it doesn't matter if the third objective isn't met exactly as described. So long as it works its fine. The gameplay mechanics have been implemented well. In reference to encouraging faster movement with a time limit, it might have been worthwhile implementing a difficulty system with different time limits to lower the entry bar. Speed felt like it was even over-encouraged. A little more time would have been nice. Otherwise it works as intended but that would have been a nice addition. The implementation [of the events system] seems fine and it seems to work. So long as the events are consistent and don't randomly stop or anything then its fine."

Is there anything in the game which you would change?

From the client: "Sometimes I felt that when jumping you travel to far and it felt a little disconcerting at times, and so a greater air resistance effect would have potentially made the movement better. A set

terminal [maximum] velocity would have also been good. Other than that I don't I would change anything else."

Discussion of Feedback from the Client

I think the clients criticisms and appraisals are fair, and I would agree that while some of the implementations of certain objectives have their issues, overall the project appears to have been successful. I feel that throughout the project I have been very critical of my own work, and I appreciate the sentiment that many of the things, primarily the collision detection, are hard to get right, and are things that even large gaming studios struggle with. I agree with the sentiment that the level construction is complicated. Some documentation could be created in order to help with this, though I do not think that this will alleviate the fact that it is still a text based system for designing levels, and will therefore be more complicated and difficult to work with. I reiterate my view that a graphical system for designing levels would be ideal, yet very difficult to implement. I would attempt to produce this feature if I were to do this project again.

In regards to the gameplay mechanics, I agree that a difficulty setting for the levels would be ideal, and is something I should have considered when first designing the gameplay. In regards to the high skill required for the game, I think that the difficulty is good and suits this style of game, though I recognise that the aforementioned difficulty setting could make the game accessible to a wider audience. I would also agree with the client that the added variation in gameplay caused by extra movement options which could have been added would add extra layers to the gameplay. To build on this, I would also mention that the difficulty of the game in its current state could have been offset by giving the player more options to complete levels, such as more advanced movement techniques. This is once again something I would definitely attempt to implement given the opportunity to do this project again. Overall, I think the clients criticisms and appraisals of the project are fairly given, and I recognise that I have historically been very critical of my project.

Closing Thoughts

I will conclude this project by referring back to my original explanation and modelling of the problem. I originally described a game with many movement techniques which require skill to execute quickly, where the intent of the game is to encourage the player to maintain high speeds while progressing through the game. I would argue that, whilst I have been unsuccessful in implementing advanced movement techniques, the simple movement that there is allows for variation in how levels are played and results fun gameplay. The result was a game which does indeed satisfy the requirement of encouraging fast movement in order to complete levels in the allocated time, which results in the player needing to focus on conserving speed, whilst also needing to have the skill to progress through the level. The options available when creating a level adds to the depth of gameplay that I was originally intending to produce using the movement techniques. Overall I believe that the final product has largely satisfied the requirements I originally set out in the analysis, on the grounds that I have produced a fun gaming experience which both the client and myself are happy with and enjoy.

References

Please refer to the file `IMPORTANT_NOTICE.TXT` for information on my code, the libraries I have used, and the other sources I have referenced. As is mentioned in the file, I have referenced my sources at the point of using the code or knowledge I retrieved.